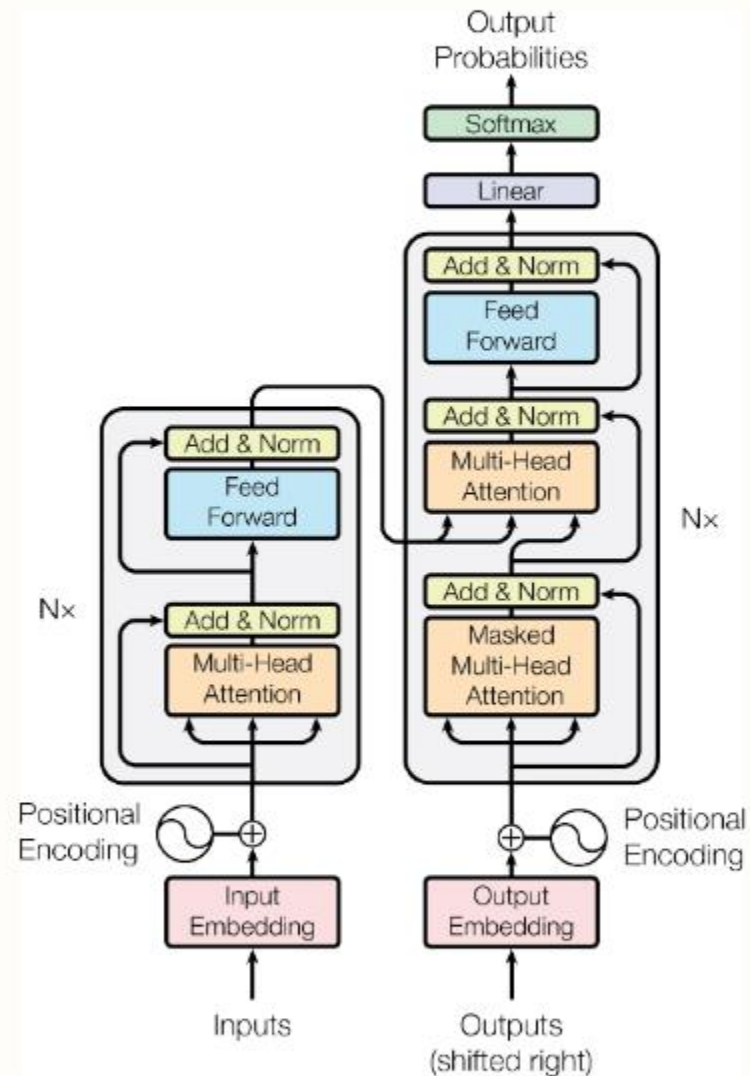


The Original Transformer Architecture

The 2017 paper assembled familiar components into an encoder-decoder system.

READ THE PUBLISHED DIAGRAM FROM BOTTOM TO TOP



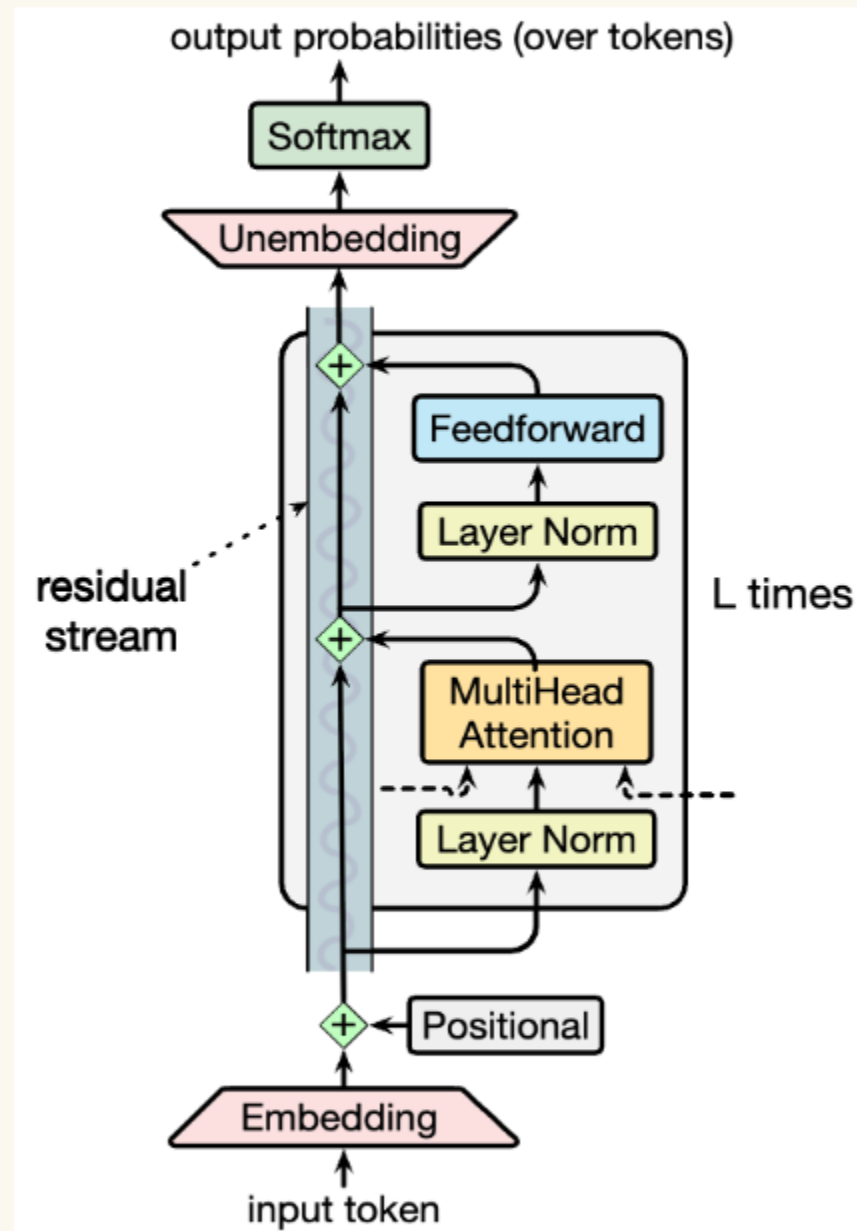
What to recognize

- 1 Encode the source sequence**
The left stack alternates self-attention and a position-wise feedforward network.
- 2 Mask the target-side self-attention**
The decoder can use earlier outputs, but it cannot look at future target tokens.
- 3 Revisit the encoder through cross-attention**
The middle decoder attention block queries the source representation at every output step.
- 4 Produce next-token probabilities**
A linear vocabulary head and softmax turn the final decoder state into a distribution.

Decoder-only LLMs keep the masked decoder stack—but remove the encoder and cross-attention.

The Decoder-Only Transformer at a Glance

Today we connect the block computation to embeddings, vocabulary prediction and generation.



READ FROM BOTTOM TO TOP

Softmax turns the scores into probabilities.

Unembedding produces one score per vocabulary token.

L blocks repeatedly update the residual stream.

Token and position vectors form the input.

We begin by completing the computation inside one block.

Lecture Overview

The lecture completes the Transformer architecture and follows its computation from token IDs to generation.

FOUR PARTS OF THE LECTURE

TRANSFORMER BLOCK COMPUTATION

— **SwiGLU feedforward computation and normalization**

Parallel computation across positions and stacked blocks

INPUT REPRESENTATIONS

— **Token IDs, embedding lookup and the embedding linear layer**

Token embeddings combined with position embeddings

POSITION AND VOCABULARY SCORES

— **Absolute and relative methods for representing position**

Language modeling head, unembedding and weight tying

MODEL SCALE AND GENERATION

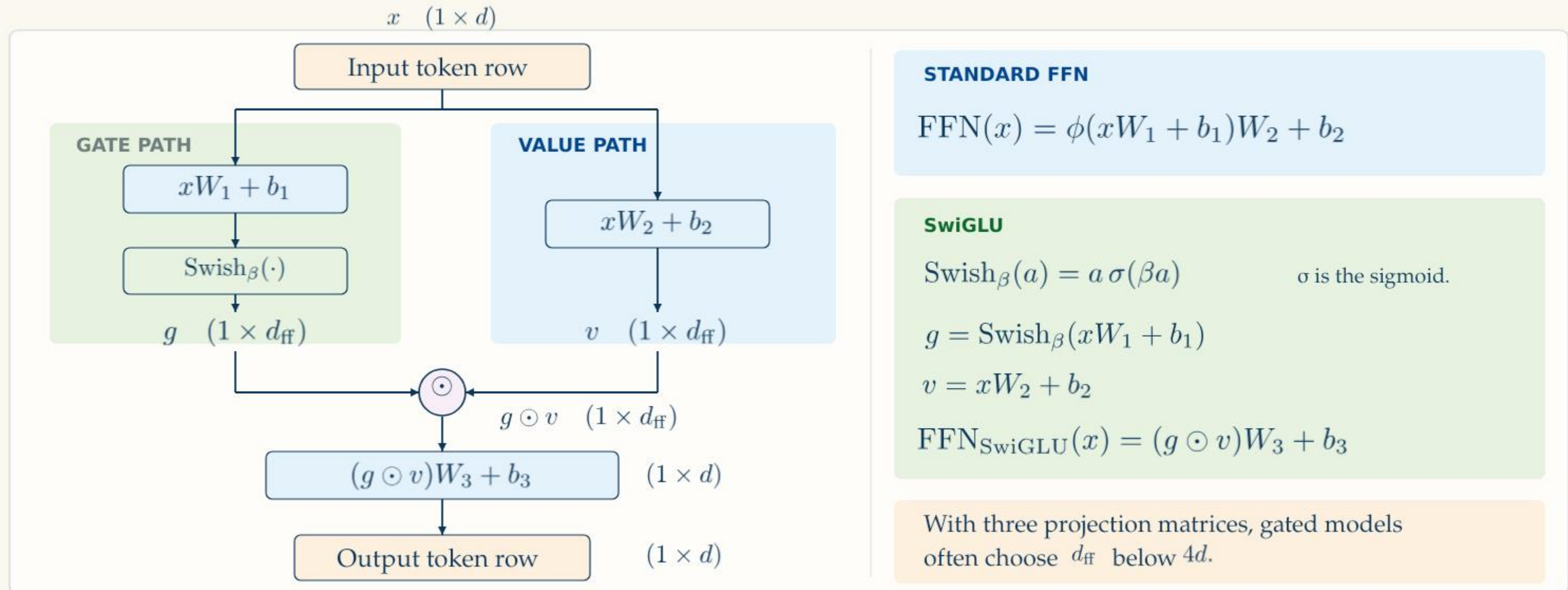
— **GPT-2 Small parameter counts and the complete forward pass**

Autoregressive generation from successive next-token distributions

We begin with the computations that complete one Transformer block.

Modern Feedforward Blocks Use SwiGLU

A learned gate controls which transformed features pass through.



Attention mixes information across token rows; the feedforward layer transforms features independently within each row.

Next: how normalization controls the scale of each token vector.

LayerNorm and RMSNorm

Both control scale; RMSNorm skips mean-centering.

LayerNorm

$$\mu = \frac{1}{d} \sum_j x_j$$

$$\sigma^2 = \frac{1}{d} \sum_j (x_j - \mu)^2$$

$$\text{LayerNorm}(x) = \gamma \odot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

centers and rescales each token vector

RMSNorm

$$\text{rms}(x) = \sqrt{\frac{1}{d} \sum_j x_j^2 + \epsilon}$$

$$\text{RMSNorm}(x) = \gamma \odot \frac{x}{\text{rms}(x)}$$

rescales each token vector without subtracting its mean

Parallel Computation in a Transformer Block

The block computes all token positions at each stage without a recurrent dependency between positions.

ATTENTION COMMUNICATES; THE OTHER OPERATIONS PROCESS ROWS IN PARALLEL

CAUSAL SELF-ATTENTION ACROSS POSITIONS

Each query combines values from its own position and earlier positions.

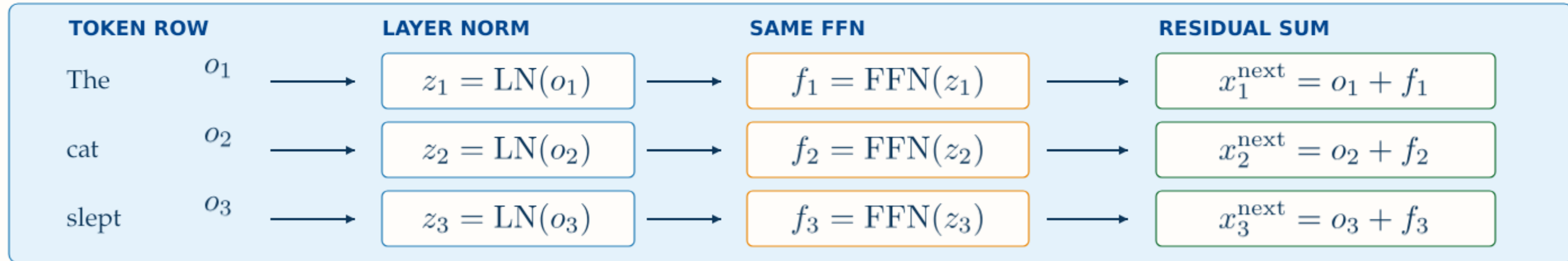
$$A = \text{softmax}_{\text{row}} \left(QK^T / \sqrt{d_k} + M \right) V$$

$$M_{ij} = -\infty \text{ for } j > i$$
$$M_{ij} = 0 \text{ otherwise}$$

AFTER THE ATTENTION RESIDUAL ADDITION

$$U = \text{MHA}_{\text{causal}}(\text{LN}(X))$$

$$O = X + U$$



The three rows can pass through each stage together; successive stages still depend on earlier results.

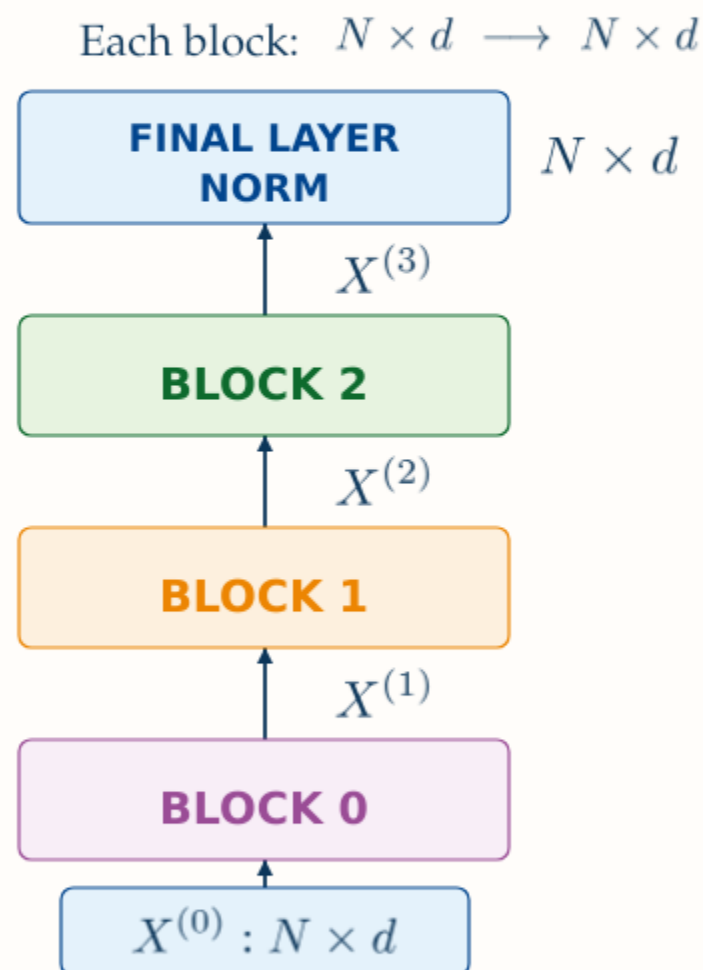
All positions in an available input sequence can be processed together.
Generating successive new tokens remains sequential.

Stacking Transformer Blocks

Each block preserves N token positions and d features, so its output enters the next block.

THE SAME COMPUTATIONAL STRUCTURE REPEATS WITH SEPARATE PARAMETERS

THE RESIDUAL STREAM THROUGH DEPTH



EXAMPLE BLOCK COUNTS

Model	Blocks
GPT-2 Small	12
GPT-2 Medium	24
GPT-2 Large	36
GPT-2 XL	48
Qwen3-32B	64
Llama 3.1-70B	80

Transformer blocks only.
Embeddings and LM head excluded.

A BLOCK AT DEPTH ℓ

ATTENTION UPDATE AND RESIDUAL ADDITION

$$O^{(\ell)} = X^{(\ell)} + \text{MHA}_{\text{causal}}^{(\ell)} \left(\text{LN}_{\text{att}}^{(\ell)}(X^{(\ell)}) \right)$$

FEEDFORWARD UPDATE AND RESIDUAL ADDITION

$$X^{(\ell+1)} = O^{(\ell)} + \text{FFN}^{(\ell)} \left(\text{LN}_{\text{ff}}^{(\ell)}(O^{(\ell)}) \right)$$

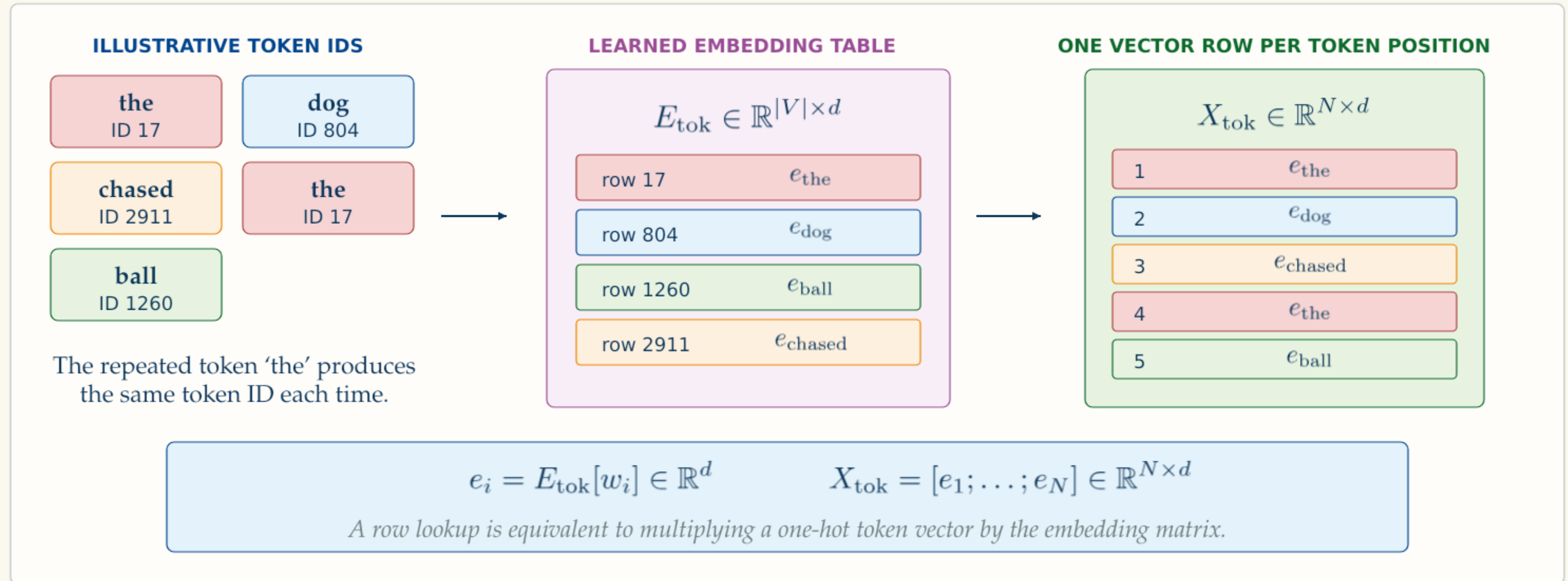
Each block learns its own attention and feedforward parameters.
The two layer norms also have separate learned gain and offset.

The first block receives vectors that carry token identity and position.

Token IDs and the Embedding Matrix

The tokenizer supplies integer IDs. The embedding matrix turns each ID into a d-dimensional vector.

A TOKEN ID SELECTS ONE LEARNED ROW



The lookup supplies token identity, but it does not yet encode where the token occurs.

Embedding as a Linear Layer

A one-hot token vector selects one row of the learned embedding matrix.

TOY EXAMPLE $|V| = 4, d = 3$

TOKEN: cat (ID 2)

$$o_i = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}$$

1×4

$\times$

EMBEDDING MATRIX

the	1	0	1
cat	0	2	1
sat	1	1	0
mat	0	1	2

$$E = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 1 \\ 1 & 1 & 0 \\ 0 & 1 & 2 \end{bmatrix}$$

4×3

$=$

OUTPUT EMBEDDING VECTOR

$$e_i = \begin{bmatrix} 0 & 2 & 1 \end{bmatrix}$$

1×3

GENERAL FORM

$$e_i = o_i E = E[w_i]$$

$(1 \times |V|)(|V| \times d) = 1 \times d$

Lookup implements this multiplication without constructing the one-hot vector.

SEQUENCE EXTENSION

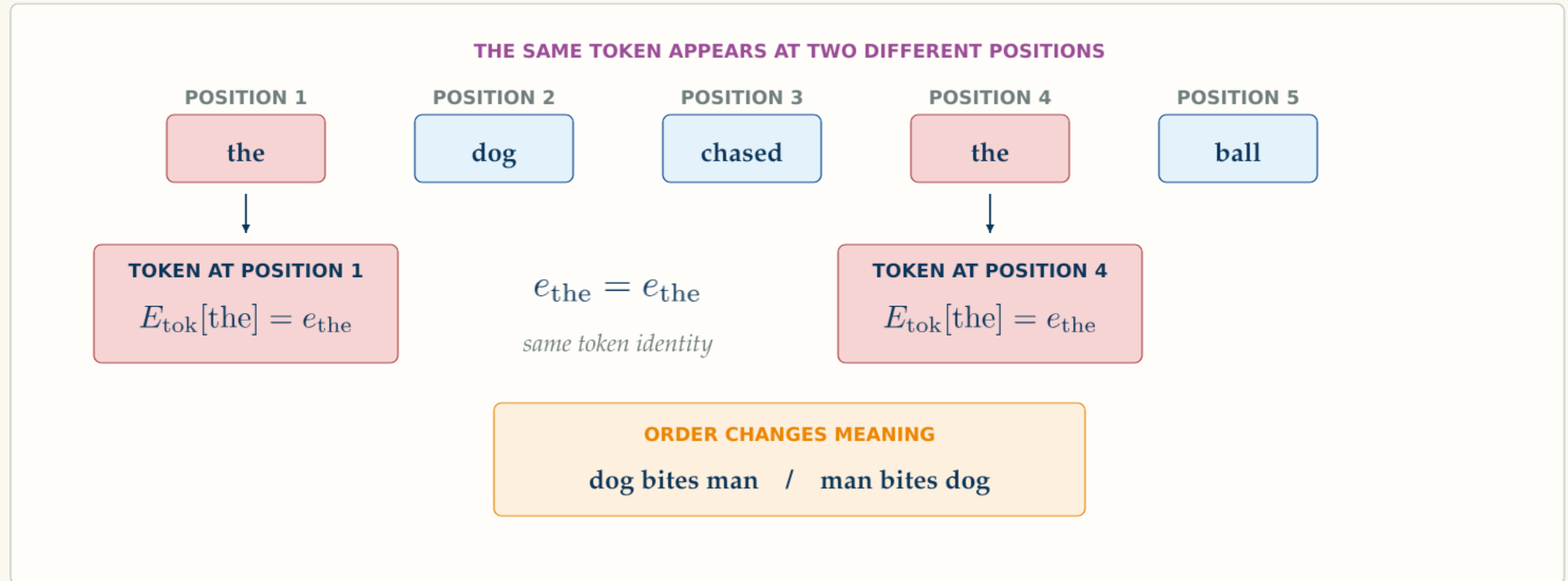
For N tokens: $X_{\text{tok}} = OE, \quad O \in \mathbb{R}^{N \times |V|}, \quad X_{\text{tok}} \in \mathbb{R}^{N \times d}$

What changes when the same token occurs at another position?

Token Embeddings and Position

A token receives the same initial embedding wherever it appears.

TOKEN EMBEDDINGS ANSWER WHAT. POSITION MUST ANSWER WHERE.



The input needs a second vector that identifies the position of each token.

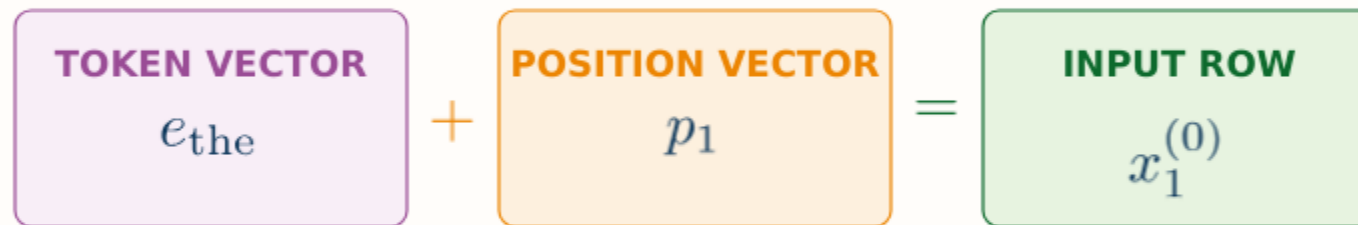
Token and Position Embeddings

Each input row combines what the token is with where it occurs.

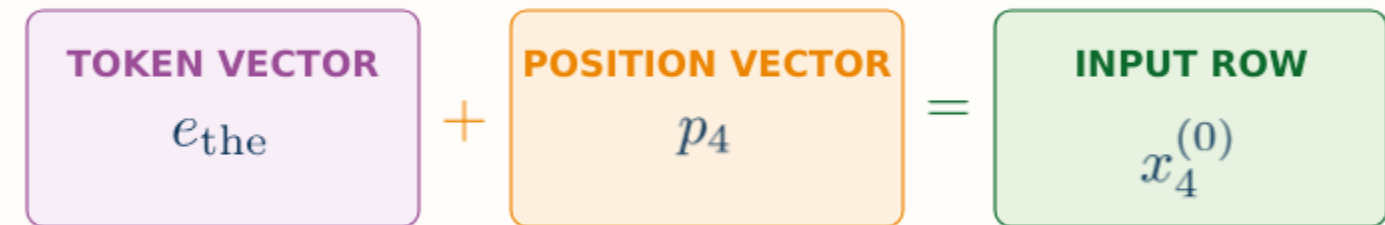
THE SUM BECOMES THE INPUT TO TRANSFORMER BLOCK 0

$$x_i^{(0)} = E_{\text{tok}}[w_i] + E_{\text{pos}}[i] \in \mathbb{R}^d$$

THE TOKEN 'THE' AT POSITION 1



THE TOKEN 'THE' AT POSITION 4



WHOLE-SEQUENCE REPRESENTATION

$$X^{(0)} = X_{\text{tok}} + P \in \mathbb{R}^{N \times d}$$

TRANSFORMER BLOCK 0

$$X^{(0)}$$

Position vectors added here may be learned or fixed (e.g., sinusoidal).

With learned absolute positions, both tables are trained. The addition has no parameters.

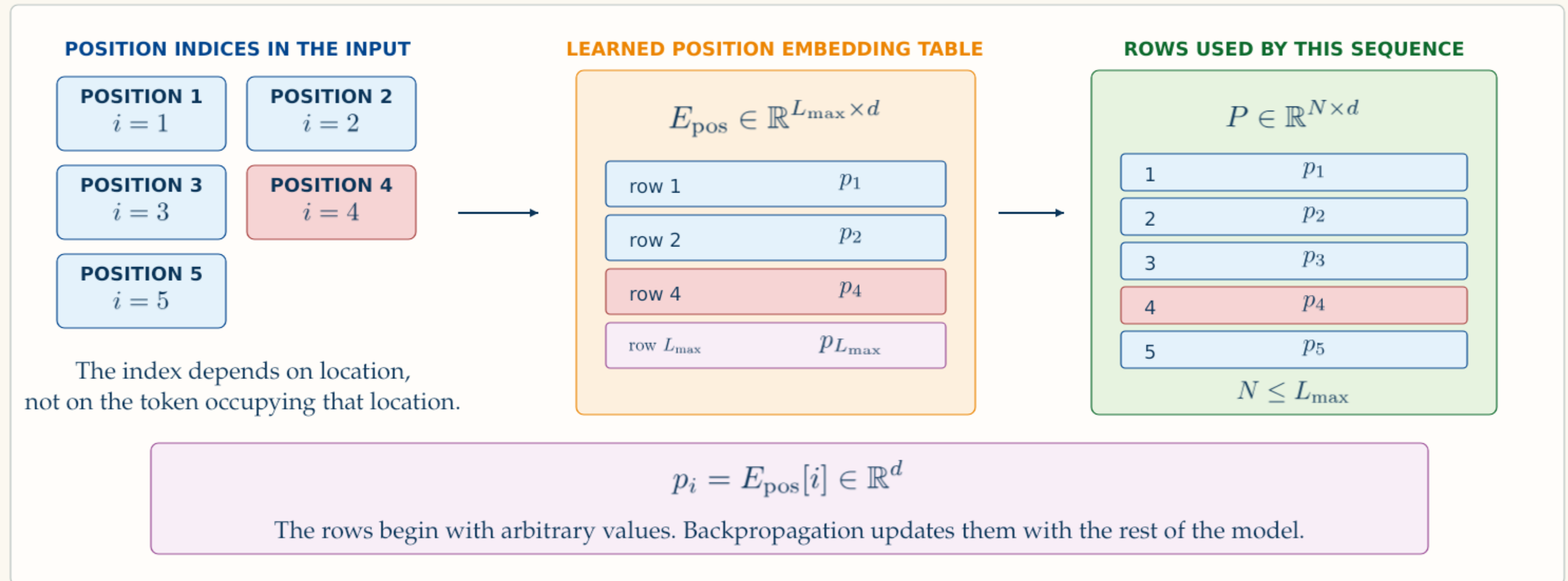
Other methods, such as RoPE, encode position within attention.

This $X^{(0)}$ is the input shown on the stacking slide.

Learned Absolute Position Embeddings

Each position index selects one d-dimensional vector that is shared across input sequences.

POSITION INDEX AND LEARNED POSITION VECTOR



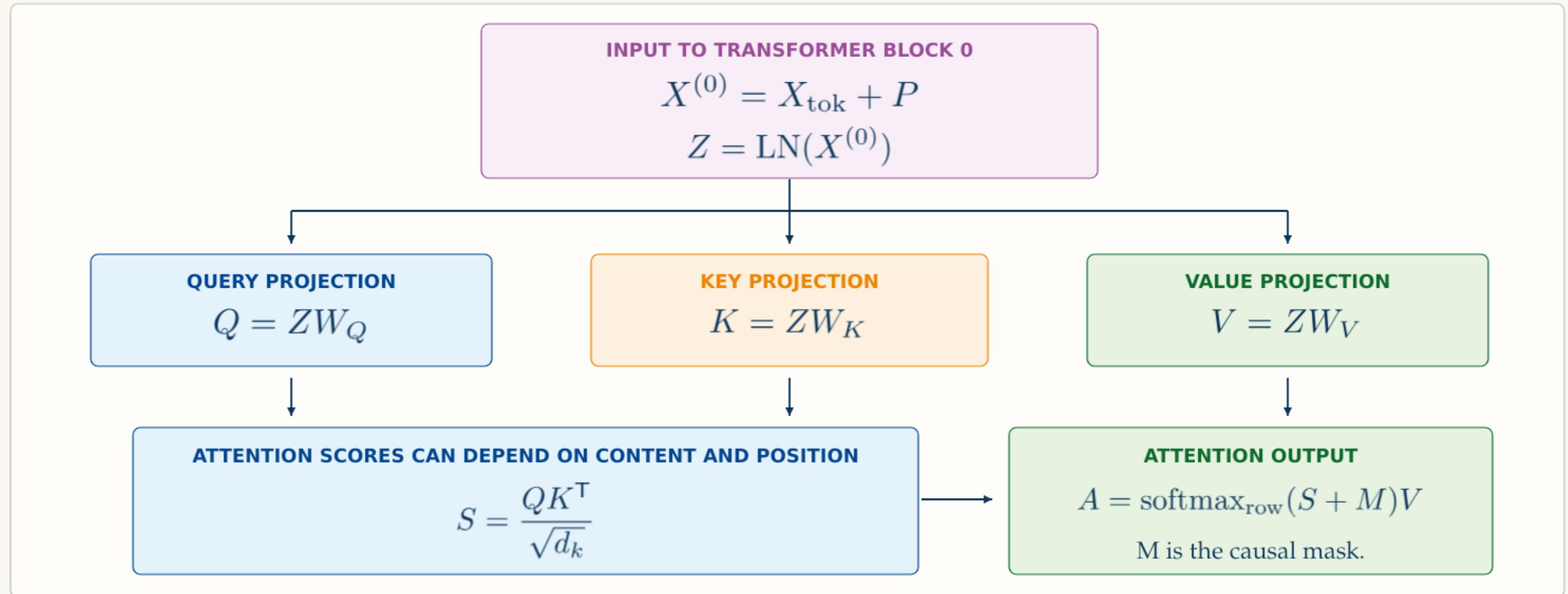
The model learns one reusable vector for each absolute position.

Vaswani et al. (2017) • Radford et al. (2018)

Position Information in Attention

The initial representation carries token and position information into the query, key and value projections.

POSITION INFORMATION IN THE QUERY, KEY AND VALUE PROJECTIONS



Position becomes usable because it changes the vectors compared by attention.

Positional Encoding Methods

Transformer architectures differ in where and how they make sequence order available.

POSITION INFORMATION CAN ENTER AT THE INPUT OR INSIDE ATTENTION

LEARNED ABSOLUTE POSITION

$$x_i^{(0)} = e_i + p_i$$

One learned vector for each position index.

Limitation: positions beyond the learned table's maximum index cannot be represented directly.

Used in the running example for this lecture.

FIXED SINUSOIDAL POSITION

$$x_i^{(0)} = e_i + s(i)$$

A fixed function generates the position vector.

No separate learned row is required for each index.

The frequencies provide structure across positions.

RELATIVE POSITION WITHIN ATTENTION

$$\Delta_{ij} = i - j$$

Position affects query-key relations or attention scores.

RoPE rotates queries and keys using their positions.

ALiBi adds a distance-dependent bias to attention scores.

These approaches make order available to attention in different ways.

How do the final token representations become vocabulary probabilities?

Language Modeling Head

The language-model head is the output layer that maps a contextual vector to one logit for every vocabulary token.

FROM ONE CONTEXTUAL VECTOR TO ONE DISTRIBUTION OVER THE VOCABULARY

FINAL POSITION AFTER LAYER NORM

$$h_N^{(L)} = [\text{LN}_{\text{final}}(X^{(L)})]_N$$
$$h_N^{(L)} \in \mathbb{R}^{1 \times d}$$

OUTPUT PROJECTION — LM HEAD

$$u_N = h_N^{(L)} W_U$$

LOGITS → SOFTMAX → PROBABILITIES

TOKEN	LOGIT	PROB.
the	5.1	0.54
a	4.3	0.24
this	3.4	0.10
...	...	...

SHAPES AND NORMALIZATION

$$W_U \in \mathbb{R}^{d \times |V|} \quad u_N \in \mathbb{R}^{1 \times |V|}$$

$$p(\cdot \mid w_{1:N}) = \text{softmax}(u_N)$$

Unlike an n-gram model's fixed window, the final contextual vector can summarize all earlier tokens within the Transformer context window.

At inference, use the final position; during training, apply the same head at every position.

Can the output projection share the input embedding weights?

Unembedding as a Linear Layer

A contextual vector produces one score for every vocabulary token.

CONTEXT VECTOR
 $h = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}$
 1×3

$\times$

UNEMBEDDING MATRIX
 $W_U = \begin{bmatrix} \text{the} & \text{cat} & \text{sat} & \text{mat} \\ 1 & 0 & 2 & -1 \\ 0 & 1 & -1 & 2 \\ 1 & -1 & 0 & 1 \end{bmatrix}$
 3×4

$=$

OUTPUT LOGITS VECTOR
 $u = \begin{bmatrix} 2 & 1 & 0 & 4 \end{bmatrix}$
 1×4
Order: the, cat, sat, mat

GENERAL FORM
 $u = hW_U \quad (1 \times d)(d \times |V|) = 1 \times |V|$

WORKED EXAMPLE: SCORE FOR mat
 $u_{\text{mat}} = 1(-1) + 2(2) + 1(1) = 4$

Each column of the unembedding matrix is a learned scoring vector for one token.

Softmax converts logits to probabilities.

Unembedding computes vocabulary scores; it does not recover the input token.

Can W_U reuse the transpose of the input embedding matrix?

Embedding Weight Tying

The same learned token vectors support input lookup and output scoring.

INPUT: LOOK UP A TOKEN VECTOR

$$|V| = 4, \quad d = 3$$

TOKEN: cat (ID 2)

$$o_{\text{cat}} = \begin{bmatrix} 0 & 1 & 0 & 0 \end{bmatrix}_{1 \times 4}$$

$\times$

EMBEDDING MATRIX

$$E = \begin{bmatrix} \text{the} & 1 & 0 & 1 \\ \text{cat} & 0 & 2 & 1 \\ \text{sat} & 1 & 1 & 0 \\ \text{mat} & 0 & 1 & 2 \end{bmatrix}_{4 \times 3}$$

$=$

TOKEN EMBEDDING

$$e_{\text{cat}} = \begin{bmatrix} 0 & 2 & 1 \end{bmatrix}_{1 \times 3}$$

$$e_{\text{cat}} = o_{\text{cat}} E = E[\text{cat}]$$

One matrix, used in two orientations.
 $|V|d$ learned weights.

$$W_U = E^T$$

Transposition reorients the shared weights;
it does not invert the Transformer.

OUTPUT: SCORE EVERY TOKEN

$$E^T = W_U \in \mathbb{R}^{d \times |V|}$$

CONTEXT VECTOR

$$h = \begin{bmatrix} 1 & 2 & 1 \end{bmatrix}_{1 \times 3}$$

$\times$

TRANPOSED MATRIX

$$E^T = W_U = \begin{bmatrix} \text{the} & 1 & 0 & 1 \\ \text{cat} & 0 & 2 & 1 \\ \text{sat} & 1 & 1 & 0 \\ \text{mat} & 0 & 1 & 2 \end{bmatrix}_{3 \times 4}$$

$=$

VOCABULARY LOGITS

$$u = \begin{bmatrix} 2 & 5 & 3 & 4 \end{bmatrix}$$

the cat sat mat

$$u = h E^T \in \mathbb{R}^{1 \times 4}$$

How much of GPT-2's parameter budget comes from embeddings, attention and FFNs?

GPT-2 Small Parameter Counts

A parameter budget connects the architecture to the size of its learned matrices.

gpt-2

Code and models from the paper "[Language Models are Unsupervised Multitask Learners](#)".

You can read about GPT-2 and its staged release in our [original blog post](#), [6 month follow-up post](#), and [final post](#).

We have also [released a dataset](#) for researchers to study their behaviors.

* Note that our original parameter counts were wrong due to an error (in our previous blog posts and paper). Thus you may have seen small referred to as 117M and medium referred to as 345M.

MODEL CONFIGURATION

12 blocks, $d = 768$, $d_{\text{ff}} = 3072 = 4d$, $|V| = 50,257$, $L_{\text{max}} = 1024$

COMPONENT	PARAMETERS	MILLIONS	SHARE
Token embeddings	38,597,376	38.60M	31.0%
Position embeddings	786,432	0.79M	0.6%
Attention matrix weights across 12 blocks	28,311,552	28.31M	22.8%
FFN matrix weights across 12 blocks	56,623,104	56.62M	45.5%
Biases + LayerNorm parameters	121,344	0.12M	0.1%
Total	124,439,808	124.44M	

TOKEN EMBEDDING TABLE

$$|V|d = 50,257 \times 768$$

38.60M parameters

ATTENTION PER BLOCK

$$d(3d) + d^2 = 4d^2 = 2.36\text{M}$$

12 blocks: 28.31M

FFN PER BLOCK

$$d(4d) + (4d)d = 8d^2 = 4.72\text{M}$$

12 blocks: 56.62M

The tied LM head adds no separate parameter matrix.

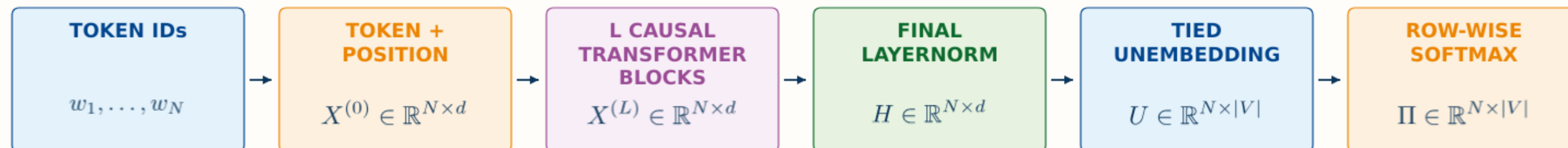
For an FFN width of $4d$, FFN matrix weights are twice attention matrix weights.
The large vocabulary also makes the token embedding table a major parameter component.

With the complete parameterized model in place, we can trace one full forward pass.

Decoder-Only Transformer Forward Pass

Token IDs become contextual states, vocabulary logits and next-token probabilities.

ONE DISTRIBUTION AT EACH POSITION



FORWARD-PASS EQUATIONS

TOKEN + POSITION

$$X^{(0)} = E_{\text{tok}}[w_{1:N}] + E_{\text{pos}}[1:N]$$

L CAUSAL BLOCKS

$$X^{(\ell+1)} = \text{Block}_{\ell}(X^{(\ell)}) \\ \ell = 0, \dots, L-1$$

FINAL LAYERNORM

$$H = \text{LN}_{\text{final}}(X^{(L)})$$

TIED UNEMBEDDING

$$U = H E_{\text{tok}}^T$$

ROW-WISE SOFTMAX

$$\Pi = \text{softmax}_{\text{row}}(U)$$

TRAINING

Every row Π_i predicts w_{i+1}

All positions train in parallel under the causal mask.

INFERENCE

The final row Π_N

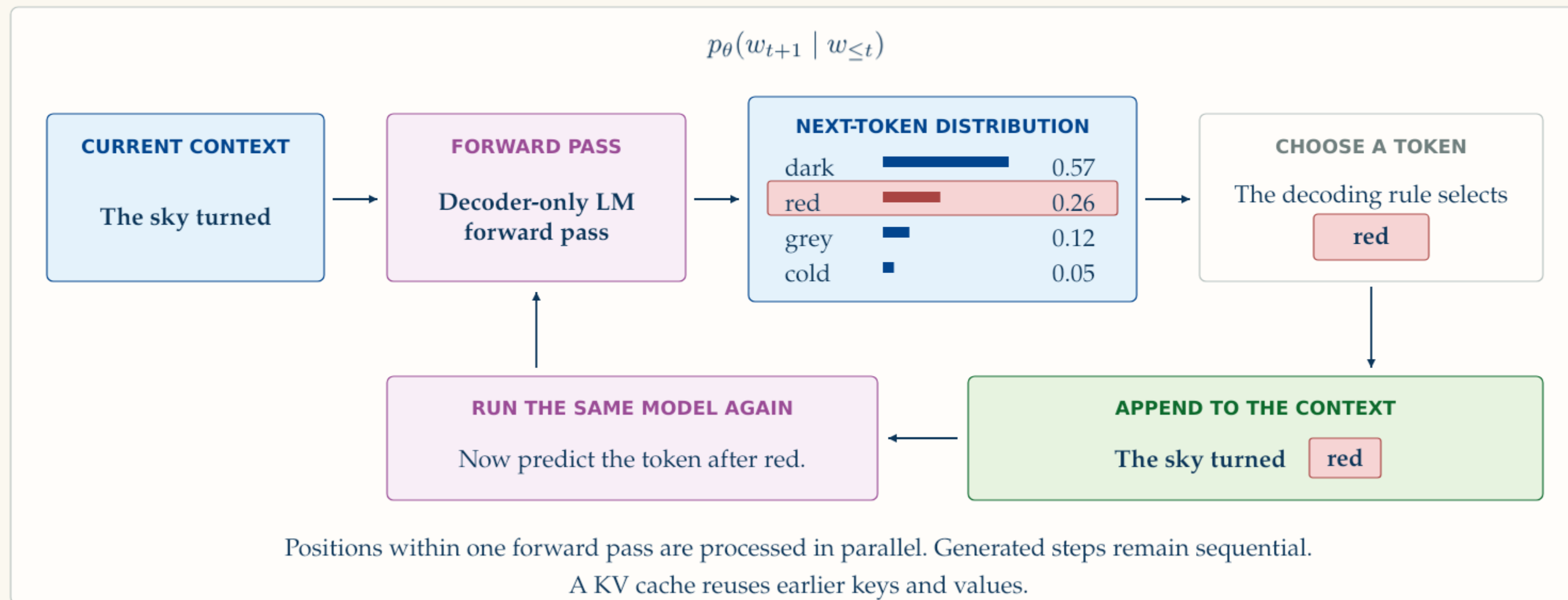
gives the distribution for the next token.

Generation selects one token, appends it to the context, and runs the model again.

Autoregressive Generation

Each generated token becomes part of the context used to compute the next distribution.

ONE FORWARD PASS PRODUCES ONE NEXT-TOKEN DISTRIBUTION



Which rule should choose the next token from this distribution?